UNIT 2 · TOPIC 2.3

Protecting Physical Spaces

Assignment, Answer Key

Teacher answer key, 41 points total.

Practice Questions

2.3.a · Physical access controls (locks, badges, biometrics, mantraps)

1. An administrator wants to prevent tailgating (where an unauthorized person follows an authorized person through a door). Which of the following physical access controls is the most effective solution for this specific threat? (1 pt)

- ○ A. A high-security biometric fingerprint scanner
- ○ B. A physical keyed deadbolt lock
- **● C. A mantrap (access vestibule) with interlocked doors**
- ○ D. An RFID badge reader with real-time audit logging

Why: A mantrap features two interlocked doors where the second door cannot open until the first door closes, ensuring that only one person (or one authenticated party) can pass through per cycle, directly preventing tailgating. Badge readers and biometrics authenticate identity but do not physically block tailgating.

2. Which of the following are distinct security advantages of an electronic RFID badge system over traditional mechanical keyed locks? (Select all that apply) (2 pts)

- **■ A. The ability to instantly revoke a user's access credentials in a central database without replacing physical hardware.**
- □ B. Complete immunity to credential cloning and signal sniffing attacks.
- **■ C. Automatic generation of an audit trail showing the timestamp and identity associated with each entry.**
- □ D. Independence from electrical power sources and network connectivity.

Why: Electronic RFID badge systems allow administrators to revoke credentials instantly in a database, rendering the physical badge useless immediately. They also log each access event, creating a digital audit trail. However, they are vulnerable to signal cloning and generally rely on power and network connectivity.

3. What is the common biometric performance metric that measures the probability that a system will incorrectly grant access to an unauthorized user? (1 pt)

Acceptable answers: false acceptance rate; far; false acceptance

Why: The False Acceptance Rate (FAR) is the probability that a biometric security system will incorrectly authorize an unauthorized person.

2.3.b · Surveillance systems (CCTV, monitoring)

4. A facility relies entirely on CCTV cameras with no guards, alarms, or locks. Which statement BEST describes the limitation of this setup? (1 pt)

- ○ A. CCTV cannot record video at night under any circumstances
- ○ B. CCTV signals cannot be viewed remotely without a VCR
- **● C. CCTV is a detective control; it can record an intrusion but cannot physically stop it**
- ○ D. CCTV always requires a dedicated coaxial cable to function

Why: Cameras document events (detective) but do not themselves prevent an intruder from breaching a fence or door; that requires preventive controls like locks and fencing plus a response process.

5. An organization replaces its analog coaxial CCTV system with networked IP cameras. What is the PRIMARY new risk introduced by this upgrade? (1 pt)

- **● A. IP cameras are now exposed to network-based attacks such as credential stuffing or firmware exploits**
- ○ B. IP cameras cannot record footage for later review
- ○ C. IP cameras require more physical cabling than analog systems
- ○ D. IP cameras cannot support motion detection or analytics

Why: Because IP cameras are addressable network devices, they inherit cyber risks (port scanning, default credentials, unencrypted RTSP sniffing, DoS) that an isolated analog coaxial loop is immune to.

6. Which of the following are recommended mitigations for logical vulnerabilities in an IP camera network? Select ALL that apply. (2 pts)

- **■ A. Isolate cameras on a dedicated VLAN separate from general office traffic**
- □ B. Leave factory default admin credentials unchanged so support technicians can log in easily
- **■ C. Encrypt video streams in transit and at rest**
- **■ D. Apply firmware updates to camera devices regularly**

Why: VLAN isolation, encryption, and firmware patching all reduce the attack surface. Leaving default credentials unchanged is the opposite of a mitigation, it is one of the most common ways attackers take over IP cameras.

7. Give the term for an area within or around a facility that is not visible to any surveillance camera. (1 pt)

Acceptable answers: blind spot; blindspot

Why: A blind spot is any unmonitored area; designers use overlapping fields of view between cameras to eliminate them.

8. A security team watches live CCTV feeds on a monitor wall and dispatches a guard the moment they see a fence breach in progress. This is an example of: (1 pt)

- ○ A. Passive monitoring
- ○ B. Anti-passback
- ○ C. Thermal imaging
- **● D. Active monitoring**

Why: Active monitoring means personnel watch live feeds in real time and can respond to an incident as it happens, unlike passive monitoring, which only reviews recorded footage after the fact.

9. Why is passive-only monitoring (recording footage for later review, with no live viewing) considered insufficient on its own for stopping an attack in progress? (1 pt)

- ○ A. Passive monitoring uses more storage than active monitoring
- ○ B. Passive monitoring requires a live guard to be present at all times
- ○ C. Passive monitoring cannot be combined with thermal cameras
- **● D. Passive monitoring only produces footage that is reviewed after an incident already occurred, so it cannot interrupt the attack itself**

Why: Passive/recorded monitoring is valuable for forensic investigation after the fact, but because no one is watching in real time, it does nothing to interrupt or prevent an attack while it is happening.

10. Name the camera type that can be remotely swept across a wide area but temporarily creates a blind spot on the side it is not currently pointed toward. (1 pt)

Acceptable answers: ptz; pan-tilt-zoom; pan tilt zoom camera

Why: A Pan-Tilt-Zoom (PTZ) camera offers wide coverage over time but, at any given instant, is only pointed at part of its range, the rest is a temporary blind spot.

2.3.c · Security guards and layered defense-in-depth

11. A guard at the reception desk denies entry to a visitor whose credentials look valid but whose story about their visit does not hold together under questioning. Which guard function does this best illustrate? (1 pt)

○ A. Patrol

○ B. Response

● **C. Reception and access verification**

○ D. Visitor logging

Why: Checking credentials and probing an inconsistent story at an entry point is the reception/access-verification role, something an automated badge reader cannot do.

12. Which of the following are examples of a guard's response function? Select ALL that apply. (2 pts)

■ **A. Physically investigating after a door sensor alarm fires**

□ B. Walking a randomized route through the facility each shift

■ **C. Responding in person when a camera detects motion in a restricted area after hours**

□ D. Recording a visitor's name and sponsor in a log at the front desk

Why: Response means physically investigating triggered alarms or detected motion. Randomized patrol routes are the patrol function, and recording visitor names is visitor-log administration, not response.

13. A facility relies only on a badge reader at its single entrance. According to the defense-in-depth principle, what is the main weakness of this setup? (1 pt)

● **A. An attacker only needs to defeat one control (a cloned or stolen badge) to succeed**

○ B. Badge readers are always unreliable hardware

○ C. Badge readers cannot log entry times

○ D. Badge readers require a network connection to function

Why: With a single control layer, defeating that one layer is sufficient for an attacker. Defense-in-depth requires multiple independent categories so no single failure grants access.

14. Why does adding a second security guard to the same reception desk NOT raise a facility's depth score? (1 pt)

○ A. Guards are never counted toward depth score

● **B. Two guards in the same personnel role duplicate an existing category rather than covering a new one**

○ C. Depth score only counts electronic controls

○ D. A second guard actually lowers the depth score

Why: Depth score counts distinct control categories, not total layers. A redundant personnel layer adds cost without closing a new gap that a different category (e.g., electronic or barrier) would close.

15. Which of the following are true about mantraps? Select ALL that apply. (2 pts)

■ **A. A mantrap uses two interlocking doors to prevent more than one person entering at a time**

□ B. A mantrap by itself eliminates the need for any guard verification

■ **C. A mantrap is most effective when paired with a guard who verifies identity on the far side**

■ **D. A mantrap is a barrier-category control**

Why: Mantraps are physical barrier controls that stop piggybacking, but they still benefit from a guard verifying identity in case of system error or ambiguous cases, they do not eliminate the value of human verification.

16. Name the practice of following an authorized person through a controlled door without presenting your own credential. (1 pt)

Acceptable answers: tailgating; piggybacking

Why: Tailgating (also called piggybacking) is entering behind an authorized person without independently badging or verifying in.

17. A visitor log records who entered a facility, their sponsor, and their arrival/departure times. Why is a visitor log alone NOT sufficient physical security? (1 pt)

○ A. Visitor logs are illegal to keep under most privacy laws

● **B. It is a deterrent and audit tool, not a barrier that stops unauthorized access in the moment**

○ C. Visitor logs cannot record timestamps accurately

○ D. Visitor logs replace the need for an escort policy entirely

Why: A visitor log creates accountability and an audit trail after the fact, but it does not physically prevent an unauthorized action from happening, it must be paired with escort policies and access controls.

2.3.d · Environmental controls (HVAC, fire suppression, power)

18. Why do data centers use clean-agent fire suppression (such as FM-200) instead of standard water sprinklers? (1 pt)

○ A. Clean-agent systems are cheaper to install than sprinklers

○ B. Water sprinklers are illegal in commercial buildings

○ C. Clean-agent systems require no maintenance

● **D. Water sprinklers destroy electronics on contact, even during a false activation**

Why: Water damages electronics immediately, and even a false sprinkler activation can write off an entire server room. Clean-agent systems extinguish fire without residue or equipment damage.

19. A total utility power outage occurs. Which device responds first, within milliseconds, to keep servers running? (1 pt)

○ A. Backup generator

● **B. UPS (Uninterruptible Power Supply)**

○ C. Managed PDU

○ D. VESDA smoke detector

Why: A UPS is a battery bank that responds in milliseconds, far faster than a generator can start, bridging the gap until the generator stabilizes or power is restored.

20. Which of the following are functions of a backup generator in a data center power design? Select ALL that apply. (2 pts)

■ **A. Provides extended runtime (hours to days) once started and stabilized**

□ B. Responds within milliseconds of an outage, faster than a UPS

■ **C. Typically starts and stabilizes within about 10 to 30 seconds**

■ **D. Is sometimes deployed in N+1 or 2N redundant configurations**

Why: Generators provide long runtime after a 10-30 second start delay, and critical facilities often run them in N+1 or 2N redundant configurations. A UPS, not a generator, is the millisecond-response layer.

21. What is the purpose of hot aisle / cold aisle containment in a server room? (1 pt)

○ A. It reduces the noise level of HVAC equipment

○ B. It replaces the need for a fire suppression system

● **C. It prevents hot exhaust air from mixing with cool intake air, improving cooling efficiency and preventing hot spots**

○ D. It increases humidity to prevent electrostatic discharge

Why: Physical containment (curtains, doors, or roof panels) keeps cool intake air and hot exhaust air separated so cooling equipment can work efficiently and hot spots do not form.

22. What does N+1 redundancy mean when applied to HVAC or power systems in a data center? (1 pt)

○ A. There is exactly enough capacity to meet current load with no margin

● **B. At least one extra unit of capacity exists beyond the minimum required, so a single unit failure does not cause an outage**

○ C. Two completely separate, independently powered facilities are built

○ D. Redundant systems are tested once during initial installation only

Why: N+1 redundancy means one extra unit of critical capacity (a spare generator, UPS, or HVAC unit) exists beyond the minimum needed, so a single failure does not cause a service outage.

23. Name the early-warning smoke detection system that continuously samples air and can detect smoke particles well before a standard ionization detector would trigger. (1 pt)

Acceptable answers: vesda; very early smoke detection apparatus; very early smoke detection

Why: VESDA (Very Early Smoke Detection Apparatus) samples air continuously, giving operators minutes of warning instead of seconds.

24. A managed PDU (Power Distribution Unit) provides which capability that a basic power strip does not? (1 pt)

○ A. It generates its own backup power during an outage

● **B. It allows remote monitoring of per-outlet current and remote power cycling of individual devices**

○ C. It cools the rack it is installed in

○ D. It suppresses fire at the rack level

Why: A managed PDU supports remote monitoring and per-outlet control, allowing staff to remotely reboot a locked-up device without physically entering the restricted room.

Coding Assignments (Python 3)

2.3.a · Physical access controls (locks, badges, biometrics, mantraps)

3 pts · auto-graded on the site

Coding problem, 2.3.a: Physical access controls (locks, badges, biometrics, mantraps)

Skill: Mitigate Risk · **Maps to:** 2.3.A · **Language:** Python 3 (pytest)

Problem

In physical security auditing, we look at the combination of active controls to identify vulnerabilities or gaps in a layered defense. You are tasked with implementing a function that takes a description of a facility's physical security inventory and returns all identified security gaps based on audit rules.

Implement `solve(data: dict[str, bool]) -> list[str]` in `starter.py`.

Input

A dictionary containing the following keys (each mapping to a boolean representing presence of that control):

- `keyed_lock`

- `badge_reader`
- `biometric_reader`
- `mantrap`
- `cctv`
- `security_guard`

Example:

```
{
    "keyed_lock": True,
    "badge_reader": True,
    "biometric_reader": False,
    "mantrap": False,
    "cctv": True,
    "security_guard": False
}
```

Output

A list of strings representing the detected security gaps, sorted alphabetically:

- `no_key_control`: triggered if both `keyed_lock` and `badge_reader` are absent.
- `no_electronic_access`: triggered if both `badge_reader` and `biometric_reader` are absent.
- `no_tailgating_prevention`: triggered if `mantrap` is absent.
- `no_visual_monitoring`: triggered if `cctv` is absent.
- `no_identity_verification`: triggered if both `security_guard` and `biometric_reader` are absent.

Examples

```
{
    "keyed_lock": True,
    "badge_reader": True,
    "biometric_reader": False,
    "mantrap": False,
    "cctv": True,
    "security_guard": False
}
# returns ["no_identity_verification", "no_tailgating_prevention"]

{
    "keyed_lock": False,
    "badge_reader": False,
    "biometric_reader": False,
    "mantrap": True,
    "cctv": False,
    "security_guard": True
}
# returns ["no_electronic_access", "no_key_control", "no_visual_monitoring"]
```

Constraints

- Deterministic, offline, standard library only.
- Must pass all cases in `tests.py`.

How to run

```
cd problem && python -m pytest tests.py -q
```

Reference solution:

```
"""Reference solution for 2.3.a. MUST pass 100% of tests.py.
Also the default 'blackbox' implementation students can swap out (see blackbox.yaml)."""
def solve(data: dict[str, bool]) -> list[str]:
    gaps = []

    # - If keyed lock is absent AND badge reader is absent: gap "no_key_control"
    if not data.get("keyed_lock", False) and not data.get("badge_reader", False):
        gaps.append("no_key_control")

    # - If badge reader is absent AND biometric is absent: gap "no_electronic_access"
    if not data.get("badge_reader", False) and not data.get("biometric_reader", False):
        gaps.append("no_electronic_access")

    # - If mantrap is absent: gap "no_tailgating_prevention"
    if not data.get("mantrap", False):
        gaps.append("no_tailgating_prevention")

    # - If CCTV is absent: gap "no_visual_monitoring"
    if not data.get("cctv", False):
        gaps.append("no_visual_monitoring")

    # - If security guard is absent AND biometric is absent: gap "no_identity_verification"
    if not data.get("security_guard", False) and not data.get("biometric_reader", False):
        gaps.append("no_identity_verification")

    return sorted(gaps)
```

2.3.b · Surveillance systems (CCTV, monitoring)

3 pts · auto-graded on the site

Coding problem, 2.3.b: Camera event log anomaly detector

Skill: Detect Attacks · **Maps to:** 2.3.A · **Language:** Python 3 (pytest)

Problem

You are building the alerting logic for the IP camera network at Crestfield Analytics. Each camera emits a stream of motion/detection events. Given the full event log (already sorted ascending by timestamp), identify every alert condition described below and return a sorted, de-duplicated list of alert strings.

This mirrors the lesson's monitoring concepts: active monitoring depends on software correctly surfacing anomalies (after-hours presence, a person lingering where they shouldn't, a "ghost motion" pattern consistent with a camera fault or someone deliberately avoiding facial capture) so a human operator or SIEM can act on them.

Alert rules

- **AFTER_HOURS:\<camera_id\>**, a person was detected (`person_detected: True`) at a camera outside that event's `authorized_hours` window. `authorized_hours` is a `(start, end)` tuple of UTC hours; an hour `h` is authorized if `start <= h < end`. The hour of an event is derived from its Unix timestamp: `hour = (ts % 86400) // 3600`.

- **GHOST_MOTION:\<camera_id\>**, 3 or more **consecutive** events on the same camera (in timestamp order) where `motion` is `True` but `person_detected` is `False`. Any event that breaks the pattern (`motion` is `False`, or `person_detected` is `True`) resets the streak count to zero. "Ghost motion" describes a camera repeatedly reporting motion with nothing

confirmed human, often a maintenance issue (foliage, a loose mount) or a deliberate attempt to trigger false fatigue in an operator.

• **CROWD_SERVER_ROOM**, more than 2 `person_detected: True` events occur in `area == "server_room"` within any 60-second sliding window (across any cameras). This is a single global alert, not per-camera.

Function contract

Implement `solve(events)` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Input

`events`: `list[dict]`, sorted ascending by `"ts"`, each dict has:

- `"camera_id"`: `str`
- `"ts"`: `int`, Unix timestamp (seconds, UTC)
- `"motion"`: `bool`
- `"person_detected"`: `bool`
- `"area"`: `str`
- `"authorized_hours"`: `tuple[int, int]`, `(start_hour, end_hour)`, both in `0..24`

Output

A **sorted, de-duplicated** `list[str]` of alert strings, e.g. `"AFTER_HOURS:CAM-01"`, `"GHOST_MOTION:CAM-02"`, `"CROWD_SERVER_ROOM"`. Empty list if no alerts.

Examples

```
events = [
    {"camera_id": "CAM-01", "ts": 1700000000, "motion": True, "person_detected": True,
     "area": "lobby", "authorized_hours": (8, 18)},   # hour 22 -> outside (8,18)
]
solve(events)  ->  ["AFTER_HOURS:CAM-01"]

events2 = [
    {"camera_id": "CAM-02", "ts": 1700035200, "motion": True, "person_detected": False,
     "area": "hallway", "authorized_hours": (8, 18)},
    {"camera_id": "CAM-02", "ts": 1700035260, "motion": True, "person_detected": False,
     "area": "hallway", "authorized_hours": (8, 18)},
    {"camera_id": "CAM-02", "ts": 1700035320, "motion": True, "person_detected": False,
     "area": "hallway", "authorized_hours": (8, 18)},
]
solve(events2)  ->  ["GHOST_MOTION:CAM-02"]
```

Constraints

- Deterministic, offline, standard library only.
- Timestamps within one camera's stream may be assumed to arrive in ascending order in the input; do not assume any ordering relationship *between* different cameras.
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Reference solution:

```
"""Reference solution for 2.3.b — Surveillance systems (CCTV, monitoring).
MUST pass 100% of tests.py.
Also the default 'blackbox' implementation students can swap out (see blackbox.yaml).

solve(events) -> sorted, de-duplicated list of alert strings.

Alert rules:
  AFTER_HOURS:<camera_id>  — person detected outside authorized_hours UTC hour range
  GHOST_MOTION:<camera_id> — 3+ consecutive motion=True, person_detected=False on same camera
  CROWD_SERVER_ROOM        — person_detected=True in area='server_room' > 2 times in any 60s window

Hour from timestamp: hour = (ts % 86400) // 3600   (UTC)
authorized_hours = (start, end): authorized if start <= hour < end
"""
from __future__ import annotations
from collections import defaultdict


def solve(events: list[dict]) -> list[str]:
    """Analyze a camera event log and return sorted, de-duplicated alert strings."""
    if not events:
        return []

    alerts: set[str] = set()

    # --- Rule 1: AFTER_HOURS ---
    for ev in events:
        if ev["person_detected"]:
            hour = (ev["ts"] % 86400) // 3600
            start, end = ev["authorized_hours"]
            if not (start <= hour < end):
                alerts.add(f"AFTER_HOURS:{ev['camera_id']}")

    # --- Rule 2: GHOST_MOTION (3+ consecutive motion=T, person=F per camera) ---
    # Events are already sorted ascending by ts; group by camera to preserve order.
    cam_events: dict[str, list[dict]] = defaultdict(list)
    for ev in events:
        cam_events[ev["camera_id"]].append(ev)

    for cam_id, cam_evs in cam_events.items():
        streak = 0
        for ev in cam_evs:
            if ev["motion"] and not ev["person_detected"]:
                streak += 1
                if streak >= 3:
                    alerts.add(f"GHOST_MOTION:{cam_id}")
                    break
            else:
                streak = 0

    # --- Rule 3: CROWD_SERVER_ROOM (>2 person_detected in server_room within any 60s window) ---
    server_ts = sorted(
        ev["ts"] for ev in events
        if ev["person_detected"] and ev["area"] == "server_room"
    )
    n = len(server_ts)
    for i in range(n):
        count = 1
```

```
        for j in range(i + 1, n):
            if server_ts[j] - server_ts[i] <= 60:
                count += 1
                if count > 2:
                    alerts.add("CROWD_SERVER_ROOM")
                    break
            else:
                break
        if "CROWD_SERVER_ROOM" in alerts:
            break

    return sorted(alerts)
```

2.3.c · Security guards and layered defense-in-depth

3 pts · auto-graded on the site

Coding problem, 2.3.c: Defense-in-depth scorer

Skill: Mitigate Risk · **Maps to:** 2.3.B · **Language:** Python 3 (pytest)

Problem

You are auditing the physical security posture of fictional facilities for Novara Data Center. Each facility supplies a list of security **layers** currently in place. Each layer belongs to exactly one **control category**: `"perimeter"`, `"personnel"`, `"electronic"`, `"barrier"`, or `"monitoring"` (see atom.md, these are the five categories a sound defense-in-depth plan should draw from). A facility's **depth score** is the number of *distinct* categories represented among its layers, not the total number of layers. Two guards on patrol (both `"personnel"`) only count once toward depth, because they do not close a new gap the way a different category would.

Implement `solve(layers)` that audits one facility's layer list and returns a report:

- `depth_score`, count of distinct valid categories present.
- `categories_present`, sorted list of the distinct valid category names present.
- `redundant_categories`, dict mapping each category that has *more than one* layer to the count of *extra* layers beyond the first (e.g. 3 personnel layers -> `{"personnel": 2}`). Categories with only one layer are omitted.
- `recommendation`, one of `"STRONG"`, `"ADEQUATE"`, `"WEAK"`, based on `depth_score`: - `depth_score >= 5` -> `"STRONG"` - `3 <= depth_score <= 4` -> `"ADEQUATE"` - `depth_score <= 2` -> `"WEAK"`

Unknown/invalid category strings (anything other than the five listed above) must be ignored entirely, they do not count toward `depth_score`, do not appear in `categories_present`, and do not count toward `redundant_categories`.

Function contract

Implement `solve(layers)` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Input

- `layers`: `list[dict]`, each dict has: - `"name"`: `str`, free-text description, e.g. `"lobby reception guard"` - `"category"`: `str`, one of `"perimeter"`, `"personnel"`, `"electronic"`, `"barrier"`, `"monitoring"` (or an invalid string, which must be ignored per above)

Output

A `dict` with keys `"depth_score"` (`int`), `"categories_present"` (`list[str]`, sorted ascending), `"redundant_categories"` (`dict[str, int]`), and `"recommendation"` (`str`).

Examples

```
layers = [
    {"name": "chain-link fence + lighting", "category": "perimeter"},
    {"name": "badge reader on server room", "category": "electronic"},
    {"name": "lobby reception guard", "category": "personnel"},
]
solve(layers)
# ->
# {
#   "depth_score": 3,
#   "categories_present": ["electronic", "perimeter", "personnel"],
#   "redundant_categories": {},
#   "recommendation": "ADEQUATE",
# }

layers2 = [
    {"name": "lobby guard", "category": "personnel"},
    {"name": "roving night patrol", "category": "personnel"},
    {"name": "badge reader", "category": "electronic"},
]
solve(layers2)
# ->
# {
#   "depth_score": 2,
#   "categories_present": ["electronic", "personnel"],
#   "redundant_categories": {"personnel": 1},
#   "recommendation": "WEAK",
# }
```

Constraints

- Deterministic, offline, standard library only.
- Empty `layers` list -> `depth_score` 0, empty list/dict outputs, `"WEAK"` recommendation.
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Reference solution:

```
"""Reference solution for 2.3.c. MUST pass 100% of tests.py.
Also the default 'blackbox' implementation students can swap out (see blackbox.yaml)."""
from typing import Dict, List

VALID_CATEGORIES = {"perimeter", "personnel", "electronic", "barrier", "monitoring"}


def solve(layers: List[Dict]) -> Dict:
    """Audit a facility's layer list and return a defense-in-depth report.

    depth_score = number of distinct valid categories present.
    categories_present = sorted list of those categories.
    redundant_categories = {category: extra_count} for categories with >1 layer.
    recommendation = STRONG (depth>=5) / ADEQUATE (3-4) / WEAK (<=2).
    Invalid category strings are ignored entirely.
    """
    counts: Dict[str, int] = {}
    for layer in layers:
        category = layer.get("category")
```

```
        if category not in VALID_CATEGORIES:
            continue
        counts[category] = counts.get(category, 0) + 1

    depth_score = len(counts)
    categories_present = sorted(counts.keys())
    redundant_categories = {c: n - 1 for c, n in counts.items() if n > 1}

    if depth_score >= 5:
        recommendation = "STRONG"
    elif depth_score >= 3:
        recommendation = "ADEQUATE"
    else:
        recommendation = "WEAK"

    return {
        "depth_score": depth_score,
        "categories_present": categories_present,
        "redundant_categories": redundant_categories,
        "recommendation": recommendation,
    }
```

2.3.d · Environmental controls (HVAC, fire suppression, power)

3 pts · auto-graded on the site

Coding problem, 2.3.d: Facility control-compliance checker

Skill: Mitigate Risk · **Maps to:** 2.3.C · **Language:** Python 3 (pytest)

Problem

You are auditing environmental controls across a facility's zones (rooms) for Crestfield Analytics. Each zone has a room type and a set of installed controls (UPS presence, HVAC redundancy level, fire suppression type, and floor-level leak sensor presence). The facility's control policy specifies, per room type, the minimum controls required. Given a list of zones and the policy, identify every zone that is **non-compliant**, missing one or more controls required for its room type.

This mirrors real facility audits: a server room without a UPS or with water-sprinkler suppression is a serious finding, while a general office is held to a much lighter standard.

Function contract

Implement `solve(zones, policy)` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Input

• `zones`: `list[dict]`, each dict has: - `"name"`: `str`, zone identifier, e.g. `"server-room-a"` - `"room_type"`: `str`, key into `policy`, e.g. `"server"`, `"closet"`, `"office"` - `"has_ups"`: `bool` - `"hvac_redundancy"`: `int`, number of *extra* HVAC units beyond the minimum needed (0 means no redundancy, a single unit) - `"suppression"`: `str`, one of `"water"`, `"clean_agent"`, `"none"` - `"has_leak_sensor"`: `bool`

• `policy`: `dict[str, dict]`, maps a `room_type` string to a requirements dict, which may contain any subset of these keys (a missing key means that control is not required for that room type): - `"require_ups"`: `bool`, if `True`, zone must have `has_ups == True` - `"min_hvac_redundancy"`: `int`, zone's `hvac_redundancy` must be >= this value - `"allowed_suppression"`: `list[str]`, zone's `suppression` must be in this list - `"require_leak_sensor"`: `bool`, if `True`, zone must have `has_leak_sensor == True` - A zone whose `room_type` is not a key in `policy` has **no requirements** and is always compliant.

Output

A **sorted list of unique zone name strings** that fail at least one applicable requirement. Empty list if every zone is compliant.

Examples

```
policy = {
    "server": {
        "require_ups": True,
        "min_hvac_redundancy": 1,
        "allowed_suppression": ["clean_agent"],
        "require_leak_sensor": True,
    },
    "office": {
        "allowed_suppression": ["water", "clean_agent"],
    },
}

zones = [
    {"name": "server-room-a", "room_type": "server", "has_ups": True,
     "hvac_redundancy": 1, "suppression": "clean_agent", "has_leak_sensor": True},
    {"name": "server-room-b", "room_type": "server", "has_ups": False,
     "hvac_redundancy": 1, "suppression": "clean_agent", "has_leak_sensor": True},
    {"name": "office-1", "room_type": "office", "has_ups": False,
     "hvac_redundancy": 0, "suppression": "water", "has_leak_sensor": False},
]

solve(zones, policy)  ->  ["server-room-b"]
# server-room-a meets every requirement for "server".
# server-room-b fails require_ups (has_ups is False).
# office-1 has no UPS/HVAC/leak-sensor requirements for "office", and water is an allowed
#   suppression type for office, so it is compliant.
```

Constraints

- Deterministic, offline, standard library only.
- A room type not present in `policy` has no requirements (always compliant).
- A zone is non-compliant if it fails ANY applicable requirement key present in its room type's policy entry.
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Reference solution:

```
"""Reference solution for 2.3.d. MUST pass 100% of tests.py.
Also the default 'blackbox' implementation students can swap out (see blackbox.yaml)."""
from typing import List, Dict, Any


def solve(zones: List[Dict[str, Any]], policy: Dict[str, Dict[str, Any]]) -> List[str]:
    """Return sorted list of zone names that are non-compliant with their room type's policy.

    A zone is non-compliant if it fails any applicable requirement key present in the
    policy entry for its room_type:
      - require_ups (bool): zone.has_ups must be True
      - min_hvac_redundancy (int): zone.hvac_redundancy must be >= this value
      - allowed_suppression (list[str]): zone.suppression must be in this list
      - require_leak_sensor (bool): zone.has_leak_sensor must be True
    A room_type absent from policy has no requirements (always compliant).
    """
    non_compliant = set()
```

```
for zone in zones:
    rules = policy.get(zone["room_type"])
    if not rules:
        continue

    if rules.get("require_ups") and not zone.get("has_ups", False):
        non_compliant.add(zone["name"])
        continue

    min_hvac = rules.get("min_hvac_redundancy")
    if min_hvac is not None and zone.get("hvac_redundancy", 0) < min_hvac:
        non_compliant.add(zone["name"])
        continue

    allowed = rules.get("allowed_suppression")
    if allowed is not None and zone.get("suppression", "none") not in allowed:
        non_compliant.add(zone["name"])
        continue

    if rules.get("require_leak_sensor") and not zone.get("has_leak_sensor", False):
        non_compliant.add(zone["name"])
        continue

return sorted(non_compliant)
```